

Bacharelado em Sistemas de Informação

Disciplina: Redes de Computadores

Relatório da Atividade Integradora

**Jogo de Adivinhação (Sorteio de Números) — Aplicação
Cliente-Servidor com Sockets TCP**

Protocolo de aplicação: SORTEIO/1.0

Integrantes do grupo:

Gabriel Rodrigues e Antônio Carlos

maio de 2026

1. Introdução

Este relatório documenta o desenvolvimento da Atividade Integradora proposta como fechamento do conteúdo de Sockets em Redes de Computadores. A atividade consistiu em propor, implementar e testar uma pequena aplicação cliente-servidor baseada em sockets TCP, definindo um protocolo de aplicação próprio para a troca de mensagens entre os processos.

O grupo optou pelo tema "Jogo de Adivinhação", no qual o servidor sorteia um conjunto de números e os clientes tentam adivinhar, em tempo real, qual ou quais números foram sorteados. A ênfase técnica recomendada para esse tema — comunicação via TCP com suporte a múltiplos clientes simultâneos — foi atendida por meio de um servidor multithread.

O restante deste documento está organizado da seguinte forma: a Seção 2 apresenta a proposta escolhida (item 10.1 da apostila); a Seção 3 descreve a arquitetura cliente-servidor; a Seção 4 documenta formalmente o protocolo de aplicação SORTEIO/1.0 (item 10.3); a Seção 5 mapeia o atendimento aos requisitos mínimos (item 10.2); a Seção 6 traz os principais trechos de código; a Seção 7 apresenta as evidências de teste, incluindo execução com múltiplos clientes simultâneos e em rede local (itens 10.2 e 10.4); a Seção 8 discute limitações e melhorias; e a Seção 9 conclui o relatório.

2. Proposta (item 10.1)

2.1 Tema escolhido

Jogo de Adivinhação — "o servidor sorteia número(s); clientes tentam adivinhar".

2.2 Descrição do problema

Ao iniciar, o servidor sorteia aleatoriamente 5 números inteiros distintos no intervalo de 1 a 100 (constante `QUANTIDADE_NUMEROS` = 5) e os mantém em memória durante toda a execução. Cada cliente que se conecta é solicitado a informar um nome de identificação (com verificação de nomes duplicados) e, em seguida, pode enviar repetidamente palpites (números inteiros) tentando acertar um dos números sorteados. Para cada palpite, o servidor responde imediatamente informando se o número foi sorteado ("Parabéns!") ou não ("Você errou!"), permitindo que o jogador continue tentando até decidir encerrar a sessão com o comando `/SAIR`.

2.3 Ênfase técnica

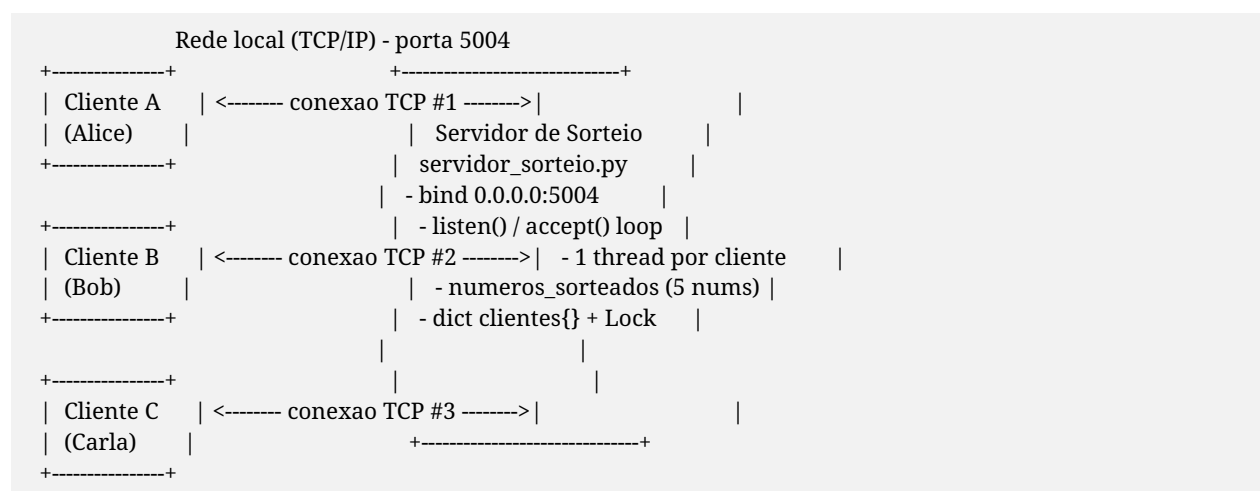
TCP com múltiplos clientes simultâneos. O servidor utiliza socket TCP (SOCK_STREAM) e atende cada conexão em uma thread independente, permitindo que vários jogadores joguem ao mesmo tempo de forma concorrente, com acesso sincronizado (via `threading.Lock`) à estrutura compartilhada que registra os jogadores conectados.

3. Arquitetura Cliente-Servidor

A aplicação é dividida em dois programas independentes, escritos em Python 3 utilizando apenas a biblioteca padrão (`socket`, `threading`, `random` e `argparse`):

- Servidor (`servidor_sorteio.py`): cria um socket TCP, faz bind em 0.0.0.0:5004 (todas as interfaces de rede da máquina) e entra em um laço infinito de `accept()`. Para cada conexão aceita, uma nova thread (daemon) é criada executando a função `atender_cliente`, responsável por todo o ciclo de vida daquele jogador. Um dicionário global `clientes` (`socket -> nome`) e um `threading.Lock` garantem o acesso seguro ao estado compartilhado entre as threads.
- Cliente (`cliente_sorteio.py`): recebe via linha de comando o endereço e a porta do servidor (argumentos `--host` e `--porta`, com valores padrão 127.0.0.1 e 5004) e estabelece uma conexão TCP. Uma thread separada (`receber_mensagens`) fica dedicada a receber e exibir, a qualquer momento, as mensagens enviadas pelo servidor, enquanto a thread principal lê a entrada do usuário (`input()`) e envia ao servidor — caracterizando comunicação full-duplex do ponto de vista do cliente.

Diagrama 1 — Visão geral da arquitetura cliente-servidor



4. Protocolo de Aplicação — SORTEIO/1.0 (item 10.3)

Esta seção documenta formalmente o protocolo de aplicação criado para o jogo de adivinhação, seguindo a estrutura sugerida na apostila (item 10.3).

4.1 Identificação geral

Elemento	Descrição
Nome do protocolo	SORTEIO/1.0
Transporte utilizado	TCP (SOCK_STREAM). Justificativa: o jogo depende de uma sequência ordenada de requisição/resposta (nome -> regras -> palpite -> resultado -> ... -> /SAIR) e cada palpite do jogador precisa de confirmação confiável do servidor. TCP garante entrega ordenada e sem perdas, eliminando a necessidade de implementar retransmissão/ACK na camada de aplicação, como seria exigido em UDP.
Porta padrão	5004/TCP
Formato das mensagens	Texto plano (UTF-8). Cada mensagem é uma string simples enviada via <code>sendall()/recv(1024)</code> ; as respostas do servidor terminam com <code>"\n"</code> para facilitar a exibição no cliente. Não há cabeçalho estruturado (ex.: JSON) — o conteúdo é interpretado diretamente como nome de jogador, palpite numérico ou comando.
Encerramento	O cliente envia o comando /SAIR (texto comparado em maiúsculas) para finalizar a sessão voluntariamente. O servidor sai do laço de atendimento, remove o jogador do dicionário compartilhado de clientes e fecha o socket. Quedas abruptas de conexão (<code>recv()</code> retornando vazio ou <code>OSError</code>) são tratadas da mesma forma, garantindo a liberação do nome do jogador. O cliente também trata <code>Ctrl+C</code> , enviando /SAIR antes de encerrar.

4.2 Comandos e mensagens

Direção	Mensagem / Comando	Quando ocorre	Resposta esperada
Servidor -> Cliente	"Digite seu nome: "	Imediatamente após a conexão ser aceita.	Cliente envia uma string com o nome do jogador.

Cliente -> Servidor	<nome do jogador> (texto livre)	Resposta ao prompt de identificação.	Se o nome já estiver em uso, servidor reenvia "Nome já em uso. Digite outro nome: " e aguarda novo envio. Caso contrário, servidor registra o jogador.
Servidor -> Cliente	"Tente adivinhar um dos 5 números sorteados entre 1 e 100! (ou /SAIR para sair)\n"	Após o nome ser aceito.	Inicia a fase de jogo; nenhuma resposta imediata é exigida.
Cliente -> Servidor	<número inteiro> (ex.: "42")	Jogador tenta um palpite durante a fase de jogo.	"Parabéns! Você acertou, X é um dos números sorteados!\n" se X estiver entre os sorteados, ou "Você errou! X não foi sorteado. Tente novamente.\n" caso contrário.
Cliente -> Servidor	/SAIR (não sensível a maiúsculas/minúsculas)	Jogador deseja encerrar a sessão.	Servidor encerra o laço, remove o jogador e fecha a conexão (sem mensagem de resposta).
Cliente -> Servidor	/ <outro comando> (ex.: "/AJUDA")	Jogador envia um comando iniciado por "/" que não é /SAIR.	"Comando desconhecido: <comando>. Comando válido: /SAIR\n"
Cliente -> Servidor	Entrada não numérica e sem "/" (ex.: "abc")	Jogador envia texto que não representa um número inteiro.	"Entrada inválida. Por favor, envie um número inteiro válido ou o comando /SAIR.\n"

4.3 Mensagens de erro previstas

Situação	Mensagem retornada pelo servidor/cliente
Nome de jogador já em uso por outra conexão ativa	"Nome já em uso. Digite outro nome: " (servidor solicita novo nome, repetidamente se necessário)

Entrada que não é um número inteiro válido e não começa com "/"	"Entrada inválida. Por favor, envie um número inteiro válido ou o comando /SAIR.\n"
Comando iniciado por "/" diferente de /SAIR	"Comando desconhecido: <comando>. Comando válido: /SAIR\n"
Cliente não consegue conectar ao servidor (porta fechada/host inacessível)	Cliente exibe: "Não foi possível conectar ao servidor em <host>:<porta>." e termina (ConnectionRefusedError tratado localmente).
Servidor encerra a conexão ou ela cai abruptamente	Cliente exibe: "[Conexão encerrada pelo servidor]" e a thread de recepção termina.

5. Requisitos Mínimos Atendidos (item 10.2)

Requisito	Como foi atendido
Pelo menos um servidor e um cliente em Python	servidor_sorteio.py e cliente_sorteio.py, ambos em Python 3.
Uso direto de sockets, sem frameworks de alto nível	Implementação utiliza apenas os módulos da biblioteca padrão socket, threading, random e argparse. Nenhum framework de rede externo foi utilizado.
Protocolo de aplicação definido e documentado	Protocolo SORTEIO/1.0 documentado na Seção 4 deste relatório e também em documentacao_protocolo.md, incluindo formato de mensagens, comandos e erros.
Tratamento de entradas inválidas ou comandos desconhecidos	Servidor trata: nomes duplicados, comandos iniciados por "/" diferentes de /SAIR e entradas não numéricas (ValueError), retornando mensagens específicas para cada caso (ver Seção 4.3). Cliente trata recusa de conexão e encerramento abrupto.
Execução em rede local (não somente localhost)	Servidor faz bind em 0.0.0.0:5004 (todas as interfaces). Cliente aceita --host e --porta por linha de comando. Teste realizado conectando ao IP da máquina na rede local (192.168.0.14), ver Seção 7.2.
Evidências de teste com pelo menos duas máquinas ou dois clientes simultâneos	Seção 7.1 apresenta um teste com três clientes (Alice, Bob e Carla) conectados simultaneamente ao mesmo servidor, com logs completos do servidor e de cada cliente.
Apresentação de limitações e possíveis melhorias	Seção 8 deste relatório.

6. Principais Trechos de Código

6.1 Servidor — configuração inicial e sorteio dos números

No início da execução, o servidor define o endereço de escuta (todas as interfaces), a porta padrão e sorteia, sem repetição, os números que os jogadores tentarão adivinhar.

```
import socket
import threading
import random

HOST = "0.0.0.0"
PORTA = 5004

clientes = {}
lock = threading.Lock()

QUANTIDADE_NUMEROS = 5
numeros_sorteados = random.sample(range(1, 101), QUANTIDADE_NUMEROS)
```

6.2 Servidor — identificação do jogador e nomes duplicados

```
conexao.sendall("Digite seu nome: ".encode("utf-8"))
nome = conexao.recv(1024).decode("utf-8").strip()

# verificação de nomes repetidos
while True:
    with lock:
        em_uso = nome in clientes.values()
    if not em_uso:
        break
    conexao.sendall("Nome já em uso. Digite outro nome: ".encode("utf-8"))
    nome = conexao.recv(1024).decode("utf-8").strip()

if not nome:
    nome = f'cliente-{endereco[1]}'

with lock:
    clientes[conexao] = nome
```

6.3 Servidor — laço do jogo, comandos e tratamento de erros

```
while True:
    dados = conexao.recv(1024)
    if not dados:
        break
    texto = dados.decode("utf-8").strip()
    if texto.upper() == "/SAIR":
        break
```



```

if texto.startswith("/"):
    conexao.sendall(
        f"Comando desconhecido: {texto}. Comando válido: /SAIR\n".encode("utf-8")
    )
    continue

try:
    chute = int(texto)
    if chute in numeros_sorteados:
        conexao.sendall(
            f"Parabéns! Você acertou, {chute} é um dos números sorteados!\n".encode("utf-8")
        )
    else:
        conexao.sendall(
            f"Você errou! {chute} não foi sorteado. Tente novamente.\n".encode("utf-8")
        )
except ValueError:
    conexao.sendall(
        "Entrada inválida. Por favor, envie um número inteiro válido "
        "ou o comando /SAIR.\n".encode("utf-8")
    )

```

6.4 Servidor — laço principal de aceitação de conexões

```

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as servidor:
    servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    servidor.bind((HOST, PORTA))
    servidor.listen(10)
    print(f"Servidor de sorteio em {HOST}:{PORTA}")
    while True:
        conexao, endereco = servidor.accept()
        threading.Thread(
            target=atender_cliente, args=(conexao, endereco), daemon=True
        ).start()

```

6.5 Cliente — parâmetros de conexão (host/porta configuráveis)

```

parser = argparse.ArgumentParser(description="Cliente para o Jogo de Sorteio")
parser.add_argument("--host", default="127.0.0.1",
                    help="Endereço IP do servidor (padrão: 127.0.0.1)")
parser.add_argument("--porta", type=int, default=5004,
                    help="Porta do servidor (padrão: 5004)")
args = parser.parse_args()

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as cliente:
    try:
        cliente.connect((args.host, args.porta))
    except ConnectionRefusedError:
        print(f"Não foi possível conectar ao servidor em {args.host}:{args.porta}.")
    return

```

6.6 Cliente — thread de recepção (comunicação full-duplex)

```
def receber_mensagens(conexao):
    while True:
        try:
            mensagem = conexao.recv(1024).decode("utf-8")
            if not mensagem:
                print("\n[Conexão encerrada pelo servidor]")
                break
            print(mensagem, end="")
        except OSError:
            break

# ...
thread_receber = threading.Thread(target=receber_mensagens, args=(cliente,))
thread_receber.daemon = True
thread_receber.start()

try:
    while True:
        mensagem = input()
        if not mensagem:
            continue
        cliente.sendall(mensagem.encode("utf-8"))
        if mensagem.upper() == "/SAIR":
            break
except KeyboardInterrupt:
    print("\nSaindo...")
    cliente.sendall("/SAIR".encode("utf-8"))
```

7. Testes Realizados e Evidências (itens 10.2 e 10.4)

Os testes a seguir foram executados em um computador com Ubuntu, com o servidor escutando em 0.0.0.0:5004. O endereço IP da máquina na rede local utilizada nos testes foi 192.168.0.14.

7.1 Teste 1 — Três clientes conectados simultaneamente

Para comprovar o suporte a múltiplos clientes simultâneos, três clientes (Alice, Bob e Carla) conectaram-se ao servidor ao mesmo tempo, cada um em sua própria thread, exercitando diferentes fluxos do protocolo: palpites errados, um palpite correto ("Parabéns!"), um comando desconhecido (/AJUDA) e uma entrada inválida ("abc"), além do encerramento via /SAIR.

Log do servidor (saída do processo servidor_sorteio.py)

```
Servidor de sorteio em 0.0.0.0:5004
<socket.socket fd=4, family=2, type=1, proto=0, laddr=('127.0.0.1', 5004), raddr=('127.0.0.1', 39922)>
[SERVIDOR] Alice entrou no jogo de sorteio.
```

```
<socket.socket fd=5, family=2, type=1, proto=0, laddr=('127.0.0.1', 5004), raddr=('127.0.0.1', 39930)>
[SERVIDOR] Bob entrou no jogo de sorteio.
<socket.socket fd=6, family=2, type=1, proto=0, laddr=('127.0.0.1', 5004), raddr=('127.0.0.1', 39944)>
[Alice] chutou o número 10
[SERVIDOR] Carla entrou no jogo de sorteio.
[Carla] chutou o número 1
[Carla] chutou o número 2
[Carla] chutou o número 3
[Carla] chutou o número 4
[Bob] chutou o número 7
[Carla] chutou o número 5
[Carla] chutou o número 6
[Carla] chutou o número 7
[Carla] chutou o número 8
[Carla] chutou o número 9
[Alice] chutou o número 25
[Carla] chutou o número 10
[Carla] chutou o número 11
[Carla] chutou o número 12
[Carla] chutou o número 13
[Carla] chutou o número 14
[Bob] chutou o número 42
[Carla] chutou o número 15
[Carla] chutou o número 16
[Carla] chutou o número 17
[Carla] chutou o número 18
[SERVIDOR] Carla saiu do jogo.
[Alice] chutou o número 50
[Alice] tentou usar comando desconhecido: /AJUDA
[Alice] enviou entrada inválida: abc
[SERVIDOR] Alice saiu do jogo.
[Bob] chutou o número 88
[Bob] tentou usar comando desconhecido: /AJUDA
[Bob] enviou entrada inválida: abc
[SERVIDOR] Bob saiu do jogo.
```

Sessão do Cliente 1 (Alice) — apenas palpites errados, comando desconhecido e entrada inválida

```
>>> Conectado ao servidor 127.0.0.1:5004
<<< Digite seu nome:
>>> Alice
<<< Tente adivinhar um dos 5 números sorteados entre 1 e 100! (ou /SAIR para sair)
>>> 10
<<< Você errou! 10 não foi sorteado. Tente novamente.
>>> 25
<<< Você errou! 25 não foi sorteado. Tente novamente.
>>> 50
<<< Você errou! 50 não foi sorteado. Tente novamente.
>>> /AJUDA
<<< Comando desconhecido: /AJUDA. Comando válido: /SAIR
>>> abc
<<< Entrada inválida. Por favor, envie um número inteiro válido ou o comando /SAIR.
>>> /SAIR
```

Sessão do Cliente 2 (Bob) — palpite correto ("Parabéns!"), comando desconhecido e entrada inválida

```
>>> Conectado ao servidor 127.0.0.1:5004
<<< Digite seu nome:
>>> Bob
<<< Tente adivinhar um dos 5 números sorteados entre 1 e 100! (ou /SAIR para sair)
>>> 7
<<< Você errou! 7 não foi sorteado. Tente novamente.
>>> 42
<<< Parabéns! Você acertou, 42 é um dos números sorteados!
>>> 88
<<< Você errou! 88 não foi sorteado. Tente novamente.
>>> /AJUDA
<<< Comando desconhecido: /AJUDA. Comando válido: /SAIR
>>> abc
<<< Entrada inválida. Por favor, envie um número inteiro válido ou o comando /SAIR.
>>> /SAIR
```

Sessão do Cliente 3 (Carla) — busca sequencial até acertar um número sorteado

```
>>> Conectado ao servidor 127.0.0.1:5004
<<< Digite seu nome:
>>> Carla
<<< Tente adivinhar um dos 5 números sorteados entre 1 e 100! (ou /SAIR para sair)
>>> 1
<<< Você errou! 1 não foi sorteado. Tente novamente.
...
>>> 17
<<< Você errou! 17 não foi sorteado. Tente novamente.
>>> 18
<<< Parabéns! Você acertou, 18 é um dos números sorteados!
>>> /SAIR
```

O log do servidor confirma o atendimento concorrente: as mensagens dos três clientes aparecem intercaladas (ex.: "[Carla] chutou o número 4" seguido de "[Bob] chutou o número 7"), evidenciando que cada conexão é processada em sua própria thread sem bloquear as demais.

7.2 Teste 2 — Acesso via rede local (não restrito a localhost)

Para validar o requisito de execução em rede local, o cliente foi executado informando o IP da própria máquina na rede (192.168.0.14), em vez de 127.0.0.1, comprovando que o servidor — que está em escuta em 0.0.0.0 — aceita conexões originadas por qualquer interface de rede e não apenas pela interface de loopback. O comando utilizado foi:

```
python3 cliente_sorteio.py --host 192.168.0.14 --porta 5004
```

Log do servidor — conexão recebida pelo endereço de rede local

```
<socket.socket fd=4, family=2, type=1, proto=0, laddr=('192.168.0.14', 5004), raddr=('192.168.0.14', 34744)>  
[SERVIDOR] TesteRede entrou no jogo de sorteio.  
[TesteRede] chutou o número 99  
[SERVIDOR] TesteRede saiu do jogo.
```

Sessão do cliente conectado via IP de rede local

```
Conectando ao servidor em 192.168.0.14:5004...  
<<< Digite seu nome:  
>>> TesteRede  
<<< Tente adivinhar um dos 5 números sorteados entre 1 e 100! (ou /SAIR para sair)  
>>> 99  
<<< Você errou! 99 não foi sorteado. Tente novamente.  
>>> /SAIR  
<<< [Conexão encerrada pelo servidor]
```

Em um cenário com duas máquinas físicas distintas na mesma rede, o procedimento seria idêntico: o servidor é executado em uma máquina (escutando em 0.0.0.0:5004) e os clientes, em qualquer outra máquina da rede, executam `cliente_sorteio.py --host <IP_DO_SERVIDOR> --porta 5004`.

8. Limitações e Possíveis Melhorias (item 10.2)

8.1 Limitações observadas

1. Protocolo textual simples sem cabeçalhos: por usar apenas texto puro em UTF-8, não há separação formal entre metadados e conteúdo, o que dificulta evoluções futuras (ex.: enviar múltiplos campos de uma vez).
2. Ausência de delimitador explícito de mensagens: como o TCP fornece um fluxo contínuo de bytes (sem preservar fronteiras de `send()`), uma chamada `recv()` pode, em tese, receber dados de mais de uma mensagem enviada em sequência muito rápida, ou parte de uma mensagem. Na prática, isso não afeta o uso interativo normal (humano digita e aguarda resposta), mas foi observado durante testes automatizados com envios em alta velocidade — reforçando a importância de, em uma versão futura, adotar um delimitador de fim de mensagem (por exemplo, `"\n"`) em ambos os sentidos.

3. Ausência de criptografia: as mensagens trafegam em texto legível; qualquer ferramenta de captura de tráfego (ex.: Wireshark) na rede local permite visualizar nomes e palpites dos jogadores.
4. Uso de locks em estrutura compartilhada: o dicionário global de clientes é protegido por threading.Lock; com um número muito grande de conexões simultâneas, esse ponto pode se tornar um gargalo de concorrência.
5. Acoplamento entre cliente e servidor: não existe um código de status/erro estruturado — o cliente apenas exibe literalmente o texto recebido do servidor, o que torna difícil tratar respostas de forma programática.

8.2 Possíveis melhorias futuras

6. Mensagens estruturadas em JSON: substituir o texto livre por mensagens estruturadas (ex.: {"tipo": "chute", "valor": 42}), separadas por quebra de linha, desacoplando a lógica do protocolo da exibição e permitindo extensões (ranking, múltiplas rodadas, etc.).
7. Criptografia via TLS/SSL: envolver os sockets com o módulo ssl da biblioteca padrão para proteger a comunicação na rede local.
8. I/O assíncrono: substituir o modelo de uma thread por cliente por asyncio, permitindo escalar para um número muito maior de jogadores simultâneos.
9. Placar/ranking persistente: registrar quantas tentativas cada jogador levou até acertar e exibir um placar geral a todos os clientes conectados.
10. Autenticação simples: exigir, além do nome, uma senha/token para evitar que um jogador assuma o nome de outro em sessões futuras.

9. Conclusão

A atividade integradora permitiu consolidar, em um único projeto prático, os principais conceitos abordados no laboratório de sockets: criação e configuração de sockets TCP, atendimento concorrente de múltiplos clientes com threads e sincronização via locks, definição de um protocolo de aplicação textual próprio (SORTEIO/1.0), tratamento de entradas inválidas e comandos desconhecidos, e execução da aplicação em rede local além do ambiente de localhost.

Os testes realizados — com três clientes simultâneos e com acesso via o endereço IP da rede local da máquina — demonstraram que o servidor atende corretamente requisições concorrentes, trata corretamente palpites corretos, incorretos, comandos desconhecidos e entradas inválidas, e encerra a sessão de forma adequada tanto em desconexões solicitadas (/SAIR) quanto abruptas.

As limitações identificadas (protocolo sem cabeçalhos estruturados, ausência de delimitador explícito de mensagens e de criptografia) representam oportunidades claras de evolução do projeto, conforme detalhado na Seção 8.